Fundamentos de Deep Learning con PyTorch

Día 1: Redes neuronales y entrenamiento

Curso introductorio para TICs y Seguridad

Dr. Rubén Hernández Torres

19 de octubre de 2026

Objetivos de aprendizaje

Al finalizar la sesión podrás:

- Explicar el funcionamiento de una red neuronal.
- Comprender el papel del gradiente.
- Identificar los componentes de PyTorch.
- Implementar una red feed-forward.
- Diferenciar clasificación y regresión.

¿Qué es Deep Learning?

Deep Learning es una rama de Machine Learning que utiliza redes neuronales con múltiples capas para aprender representaciones de los datos.

Conceptos principales:

- Parámetros aprendibles.
- Transformaciones no lineales.
- Funciones de pérdida.
- Optimización mediante gradientes.

Deep Learning en Redes, Sistemas y Seguridad

Las infraestructuras informáticas generan grandes cantidades de información:

- Tráfico de red y flujos de comunicación.
- Registros de eventos de sistemas y aplicaciones.
- Métricas de CPU, memoria y almacenamiento.
- Tiempos de respuesta y disponibilidad de servicios.
- Eventos de autenticación y alertas de seguridad.

¿Por qué utilizar Deep Learning?

Las redes neuronales pueden aprender relaciones complejas entre variables para realizar predicciones, clasificaciones y detección de anomalías.

Problemas y aplicaciones

- **Redes de computadoras**
  - Predicción de latencia y congestión.
  - Clasificación de tráfico.
  - Estimación de demanda de ancho de banda.
- **Administración de sistemas**
  - Predicción de utilización de recursos.
  - Detección de fallos y anomalías.
  - Planificación de capacidad.
- **Ciberseguridad**
  - Detección de intrusiones.
  - Identificación de phishing.
  - Análisis de eventos sospechosos.

Aplicaciones en redes de computadoras

- Clasificación de tráfico de red.
- Predicción de latencia y throughput.
- Identificación de patrones de congestión.
- Detección de anomalías en flujos de comunicación.
- Predicción de demanda de ancho de banda.

Ejemplo: predicción de latencia

Un modelo puede estimar el tiempo de ida y vuelta de una comunicación a partir de características de la red:

$$\widehat{\mathrm{RTT}} = f_\theta(x_1, x_2, \ldots, x_n)$$

Donde las entradas pueden incluir carga de tráfico, tasa de paquetes y mediciones de pérdida.

Aplicaciones en administración de sistemas

Fuentes de información:

- Utilización de CPU y memoria RAM.
- Operaciones de entrada y salida.
- Métricas de almacenamiento.
- Tiempos de respuesta de aplicaciones.
- Registros de eventos y errores.

Ejemplo: predicción de carga de CPU

Utilizando una ventana de observaciones anteriores:

$$\widehat{CPU}_{t+1} = f_\theta(CPU_{t-k}, \ldots, CPU_t)$$

La predicción puede apoyar decisiones de escalamiento y asignación de recursos.

Aplicaciones en ciberseguridad

- Sistemas de detección de intrusiones (IDS).
- Detección de anomalías en tráfico y registros.
- Clasificación de phishing y spam.
- Identificación de software malicioso.
- Detección de comportamientos sospechosos.

Ejemplo: clasificación de correos

Un modelo recibe el contenido de un mensaje y estima la clase correspondiente:

$$\text{Correo} \longrightarrow f_\theta \longrightarrow \begin{cases} \text{Legítimo} \\ \text{Malicioso} \end{cases}$$

Arquitecturas y aplicaciones

- **MLP:** Regresión y clasificación de características numéricas. Ejemplo: rendimiento de hardware.
- **CNN:** Extracción de patrones locales. Ejemplo: análisis de representaciones visuales de malware.
- **RNN, LSTM y GRU:** Procesamiento de secuencias. Ejemplo: análisis de eventos de autenticación.
- **Transformers:** Modelado contextual de texto y secuencias. Ejemplo: clasificación de mensajes y análisis de logs.
- **Autoencoders:** Aprendizaje de representaciones. Ejemplo: detección de anomalías.
- **GNN:** Procesamiento de información estructurada como grafos. Ejemplo: relaciones entre dispositivos de una red.

Retos de Deep Learning en producción

- **Calidad de los datos:** registros incompletos, ruido y etiquetas incorrectas.
- **Desbalance de clases:** los eventos maliciosos pueden ser poco frecuentes.
- **Generalización:** las condiciones de operación cambian con el tiempo.
- **Costo computacional:** memoria, latencia y consumo energético.
- **Interpretabilidad:** comprender por qué se genera una alerta.
- **Seguridad del modelo:** manipulación de entradas y evasión de detectores.

Evaluación de modelos

- **Regresión:** MAE, MSE y RMSE.
- **Clasificación:** Precision, Recall y F1-score.
- **Operación:** latencia de inferencia, memoria y throughput.
- **Seguridad:** falsos positivos y falsos negativos.

Pregunta para discusión

¿Qué es más costoso en un sistema de detección de intrusiones: generar una falsa alarma o no detectar un ataque real?

La respuesta depende del entorno, del riesgo y del costo de respuesta a incidentes.

¿Todos los problemas necesitan Deep Learning?

- Algunos problemas pueden resolverse mediante reglas, umbrales o modelos estadísticos sencillos.
- Deep Learning puede ser útil cuando existen relaciones complejas entre múltiples variables.
- Una arquitectura más grande no garantiza mejores resultados.
- La selección del modelo debe considerar precisión, costo computacional e interpretabilidad.

Idea central

No se trata de utilizar la red neuronal más compleja, sino de seleccionar una solución adecuada al problema.

La neurona artificial

Una neurona calcula una combinación lineal seguida de una función de activación:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$y = \phi(z)$$

Donde:

- x_i: características de entrada.
- w_i: pesos aprendibles.
- b: sesgo.
- ϕ: función de activación.

Redes feed-forward

Una red feed-forward transmite información desde la entrada hacia la salida.

$$\mathbf{h}_1 = \phi(W_1\mathbf{x} + \mathbf{b}_1)$$

$$\hat{\mathbf{y}} = W_2\mathbf{h}_1 + \mathbf{b}_2$$

- Capa de entrada.
- Capas ocultas.
- Capa de salida.

Pregunta: ¿Qué ocurriría si todas las capas fueran únicamente lineales?

Funciones de activación

ReLU

$$\text{ReLU}(x) = \text{máx}(0, x)$$

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Tanh

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Las activaciones no lineales permiten representar relaciones más complejas.

¿Cómo aprende una red neuronal?

El entrenamiento busca parámetros que minimicen una función de pérdida:

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta)$$

Proceso:

1. Calcular predicciones.
2. Evaluar la pérdida.
3. Calcular gradientes.
4. Actualizar parámetros.
5. Repetir durante varias épocas.

Descenso del gradiente

La actualización básica es:

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta \mathcal{L}$$

Donde:

- η: tasa de aprendizaje.
- $\nabla_\theta \mathcal{L}$: gradiente.

Discusión

¿Qué sucede si la tasa de aprendizaje es demasiado grande o demasiado pequeña?

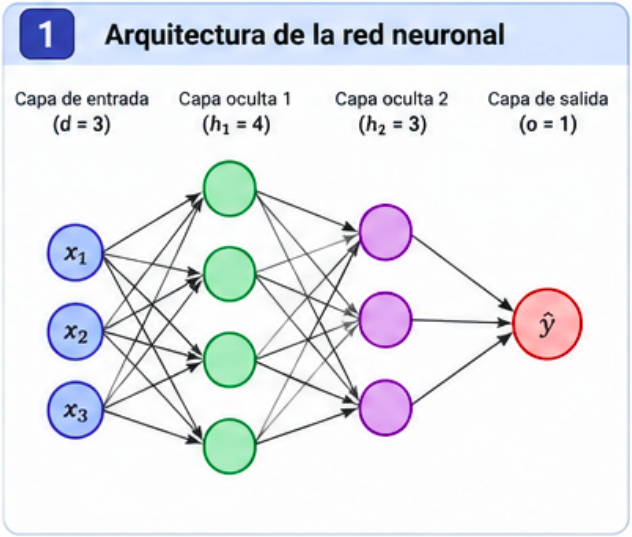
Backpropagation

Backpropagation calcula las derivadas de la pérdida respecto a los parámetros utilizando la regla de la cadena.

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w}$$

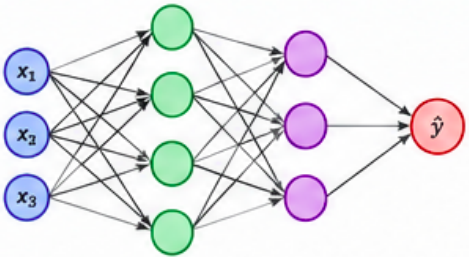
PyTorch

El motor de diferenciación automática se conoce como `autograd`.


1
Arquitectura de la red neuronal
Capa de entrada
(d = 3)
Capa oculta 1
(h_1 = 4)
Capa oculta 2
(h_2 = 3)
Capa de salida
(o = 1)
x_1
x_2
x_3
$\hat{y}$

2 Inicialización de parámetros

Pesos y sesgos inicializados (por ejemplo, aleatoriamente)



$W^{(1)} \in \mathbb{R}^{4\times3}$
$b^{(1)} \in \mathbb{R}^4$

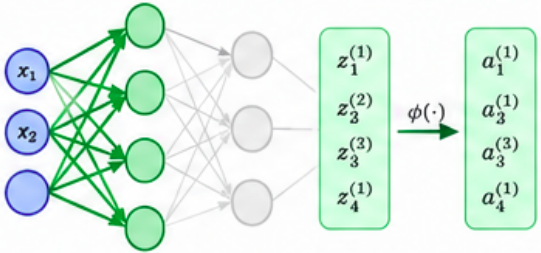
$W^{(2)} \in \mathbb{R}^{3\times4}$
$b^{(2)} \in \mathbb{R}^3$

$W^{(3)} \in \mathbb{R}^{1\times3}$
$b^{(3)} \in \mathbb{R}^1$

3 Paso forward: capa oculta 1

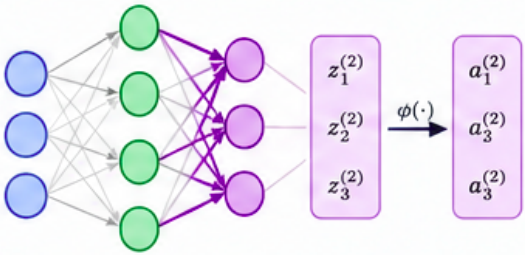
$$z^{(1)} = W^{(1)}x + b^{(1)}$$

$$a^{(1)} = \varphi(z^{(1)})$$



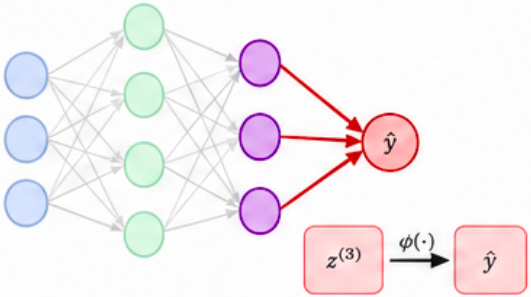
4 **Paso forward: capa oculta 2**

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \qquad a^{(2)} = \varphi(z^{(2)})$$



5 **Paso forward: capa de salida**

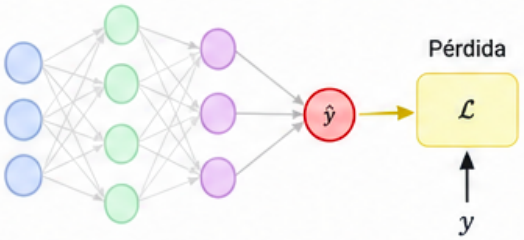
$$z^{(3)} = W^{(3)}a^{(2)} + b^{(3)} \qquad \hat{y} = \varphi(z^{(3)})$$



6 Cálculo de la pérdida

Con el valor objetivo y (por ejemplo, regresión)

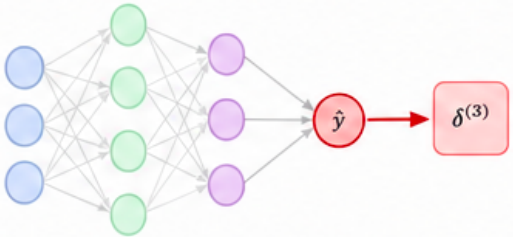
$$\mathcal{L}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$$



7 **Backprop: gradiente en la salida**

Gradiente de la pérdida respecto a la activación de salida (para MSE y activación lineal)

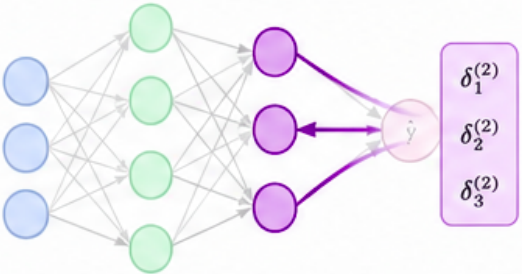
$$\delta^{(3)} = \frac{\partial \mathcal{L}}{\partial z^{(3)}} = (\hat{y} - y) \cdot \varphi'(z^{(3)})$$



8 **Backprop: gradiente en capa oculta 2**

Propagamos el error a la capa oculta 2

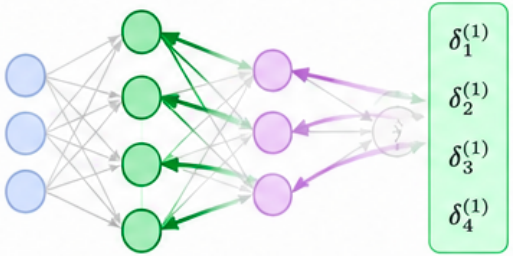
$$\delta^{(2)} = (W^{(3)})^T \delta^{(3)} \odot \varphi'(z^{(2)})$$



9 **Backprop: gradiente en capa oculta 1**

Propagamos el error a la capa oculta 1

$$\delta^{(1)} = (W^{(2)})^T \delta^{(2)} \odot \varphi'(z^{(1)})$$



10 Gradientes de los parámetros

Gradientes para cada capa l

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)}\right)^T \qquad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$$

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} \quad \frac{\partial \mathcal{L}}{\partial b^{(1)}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} \quad \frac{\partial \mathcal{L}}{\partial b^{(2)}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{(3)}} \quad \frac{\partial \mathcal{L}}{\partial b^{(3)}}$$

11 Actualización de parámetros

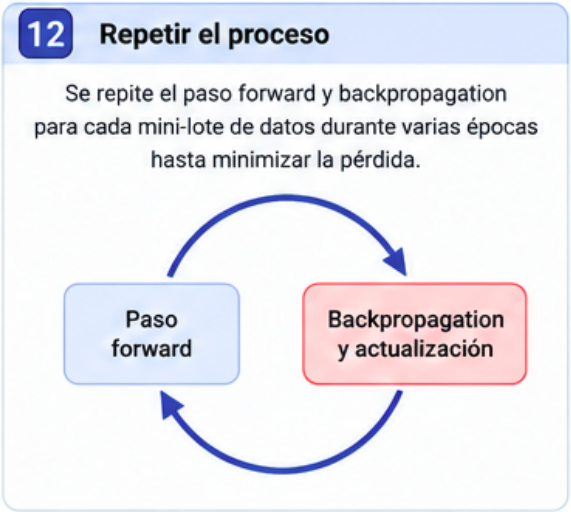
Descenso de gradiente (tasa de aprendizaje η)

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}}$$

$$b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial b^{(l)}}$$

Se actualizan todos los pesos y sesgos:

$W^{(1)}, b^{(1)}$ $W^{(2)}, b^{(2)}$ $W^{(3)}, b^{(3)}$


12
Repetir el proceso
Se repite el paso forward y backpropagation para cada mini-lote de datos durante varias épocas hasta minimizar la pérdida.
Paso forward
Backpropagation y actualización

Tensores en PyTorch

Los tensores son estructuras numéricas multidimensionales.

```
import torch

x = torch.tensor([1., 2., 3.])

A = torch.rand(3, 4)

print(x.shape)
print(A.shape)
```

Conceptos:

- Shape y dimensiones.
- Tipos de datos.
- CPU y GPU.
- Operaciones vectorizadas.

Diferenciación automática

```
import torch

x = torch.tensor(2.0,
requires_grad=True)

y = x**2 + 3*x

y.backward()

print(x.grad)
```

$$f(x) = x^2 + 3x$$

$$f'(x) = 2x + 3$$

Dataset y DataLoader

```
from torch.utils.data import (
TensorDataset, DataLoader
)

dataset = TensorDataset(X, y)

loader = DataLoader(
dataset,
batch_size=32,
shuffle=True
)
```

- Dataset: acceso a las muestras.
- DataLoader: generación de minibatches.
- Epoch: recorrido del conjunto de datos.

Clasificación y regresión

Clasificación

- Predice categorías.
- Salida: logits por clase.
- CrossEntropyLoss.
- Accuracy, precision y recall.

Regresión

- Predice valores continuos.
- Salida: valores numéricos.
- MSELoss o L1Loss.
- MAE y RMSE.

Ejemplo: clasificar tráfico sospechoso frente a predecir el consumo de ancho de banda.

Conclusiones

- Las redes neuronales aprenden parámetros.
- El gradiente dirige la optimización.
- PyTorch automatiza la diferenciación.
- Dataset y DataLoader organizan los datos.
- La evaluación mide la generalización.

Siguiente sesión

¿Cómo representar texto y secuencias para procesarlos mediante redes neuronales?